

TECHNICAL INTERVIEW PREPARATION

DevRev — Technical Round

Sourav | Senior FDE Candidate | Prepared August 2026

Known format (from candidate reviews): Screening → 2 Technical Interviews → 1–2 Behavioral. At least one technical round has included a LeetCode Medium on arrays. Round content overlaps heavily with DevRev's own product surface — an AI-native agent/dev platform — so API integration, JSON transformation, and agent tool-calling loops are all realistic in addition to pure DSA.

Contents

- 1. API Integration — Pagination, Retries, Rate Limiting
- 2. Data Transformation — Nested JSON to Normalized Schema
- 3. Agent Tool-Calling Loop
- 4. Core DSA — Arrays, Graphs, Trees, Hash Maps, Queues
- 5. Suggested Preparation Order
- 6. Quick-Reference Checklist

1. API Integration

Pagination, retries, and rate limiting — core plumbing for any FDE/integration-heavy role. Expect both conceptual questions and live coding.

1.1 Pagination

- Cursor-based vs. offset-based pagination — tradeoffs and failure modes
- What breaks with offset pagination when records are inserted/deleted concurrently
- Handling APIs with inconsistent or missing 'next page' tokens
- Writing `fetch_all_pages()` that loops until `next_cursor` is null

1.2 Retries & Backoff

- Exponential backoff with jitter — and why jitter matters with concurrent workers
- Distinguishing retryable errors (429, 503) from non-retryable (4xx client errors)
- Setting a sane max-retry cap and total timeout budget
- Idempotency — handling webhooks that arrive out of order or duplicated

1.3 Rate Limiting

- Token bucket algorithm — implement from scratch
- Leaky bucket vs. token bucket — when each is appropriate
- Coordinating a shared rate limit across multiple concurrent workers/threads
- Designing a client wrapper that respects a fixed requests-per-minute ceiling

Practice tip: Be ready to write `fetch_all_pages()`, a token-bucket rate limiter class, and a retry-with-backoff decorator/wrapper live, not just describe them.

2. Data Transformation

Parsing and normalizing nested JSON payloads into clean, relational schemas.

2.1 Flattening Nested Structures

- Flatten arbitrarily nested JSON into dot-notation keys (e.g., `customer.address.city`)
- Convert a nested payload (`ticket` → `conversation` → `messages` → `attachments`) into normalized tables with foreign keys
- Handling arrays-of-objects vs. arrays-of-primitives during flattening

2.2 Schema Handling

- Managing schema drift — a field that was a string in v1 becomes an object in v2
- Designing a transform layer that tolerates missing or renamed keys
- Null vs. missing key semantics — deciding defaults safely

2.3 Data Cleaning

- Normalizing inconsistent date/timestamp formats across sources into ISO 8601
- Deduplicating records using a stable hash of business keys when IDs are unreliable
- Validating transformed output against an expected target schema

3. Agent Tool-Calling Loop

Implementing the core loop: query $\rightarrow$ tool selection $\rightarrow$ call $\rightarrow$ observation $\rightarrow$ synthesis.

3.1 Core Loop Mechanics

- Implement a minimal ReAct-style loop given a tool registry (name, description, function)
- Route a user query to the correct tool based on intent
- Synthesize a final answer from one or more tool observations
- Add a max-iteration guard to prevent infinite loops

3.2 Robustness

- Handling a tool call that returns an error — retry, fallback tool, or escalate to user
- Memoizing repeated tool calls with identical arguments within a session
- Disambiguating when two tools could both plausibly answer the same query
- Adding a confirmation gate before any 'destructive' tool executes (e.g., delete, close, refund)

3.3 Design Considerations

- State management across multiple turns of a conversation
- Logging/observability for tool calls (what was called, with what args, what came back)
- Cost/latency tradeoffs of parallel vs. sequential tool calls

4. Core DSA

Confirmed: at least one round includes a LeetCode Medium on arrays. Prioritize array patterns, then round out with graph/tree/hash map/queue patterns relevant to workflow orchestration.

4.1 Arrays — Highest Priority (Confirmed Format)

- Two Sum / Two Sum II (sorted array, two-pointer variant)
- Subarray sum equals K (prefix sum + hash map)
- Longest substring without repeating characters (sliding window)
- Merge intervals
- Product of array except self
- Kadane's algorithm (maximum subarray sum)
- Sliding window maximum (monotonic deque)
- Find rotation point / search in rotated sorted array (binary search)

4.2 Graphs & Trees — Workflow Orchestration Patterns

- Topological sort (Kahn's algorithm, BFS-based) — task dependency ordering
- Cycle detection in a directed graph
- BFS/DFS to find all downstream nodes affected by a failure
- Build a tree from a flat list of {id, parent_id} records
- Lowest Common Ancestor (LCA) of two nodes
- Serialize / deserialize a nested tree structure

4.3 Hash Maps

- Design a task-state tracker: $O(1)$ get/set of task_id $\rightarrow$ status
- Group anagrams / group records by key
- Deduplicate using a hash map of composite keys
- LRU cache implementation (hash map + doubly linked list)

4.4 Queues & Scheduling

- BFS-based level-order execution ('wave' scheduling of parallel-safe tasks)
- Priority queue — schedule tasks by deadline/SLA
- Stack-based backtracking for partially-failed workflows

5. Suggested Preparation Order

Priority	Focus Area	Why It Matters
1	Array patterns (sliding window, two-pointer, prefix sum, Kadane's)	Confirmed format from actual DevRev candidate reviews — highest ROI
2	Topological sort + cycle detection	Matches the 'workflow orchestration' language in the JD; graph DAG questions are a natural fit for DevRev's platform
3	Tool-calling loop, coded from scratch	Maps directly to DevRev's AI-native agent product surface
4	Pagination + retry/backoff	Standard senior-engineer filter question; cheap to prepare, high likelihood
5	JSON flattening + schema drift handling	Quick win, commonly asked in integration-heavy interviews
6	Hash maps (LRU cache, dedup, grouping)	Rounds out DSA coverage if time permits
7	Trees / LCA	Lower probability but low prep cost given tree exposure from prior interview loops

6. Quick-Reference Checklist

Before the interview

- Re-solve 2–3 array problems per pattern (sliding window, two-pointer, prefix sum) under time pressure
- Hand-write Kahn's algorithm and a cycle detection routine without looking it up
- Code a tool-calling loop skeleton from memory (registry, dispatch, max-iteration guard)
- Recall the retry/backoff-with-jitter pattern and be able to explain why jitter matters
- Practice explaining tradeoffs out loud, not just producing correct code — DevRev's process has flagged 'needed hints' as a negative signal in past feedback

During the interview

- State your approach and complexity before coding
- Narrate edge cases explicitly (empty input, single element, cycles, duplicate keys)
- For system-flavored questions, connect back to real experience (Databricks PS delivery, agentic AI work) where natural

End of document.